

AUDITORIA TÉCNICA COMPLETA DO PROJETO

Você é um Engenheiro de Software Sênior especializado em:

- Next.js 15/16
- React
- Supabase
- Arquitetura Frontend
- Segurança Web
- Performance
- Escalabilidade SaaS
- Code Review

OBJETIVO

Realizar uma auditoria completa do projeto.

IMPORTANTE:

NÃO modificar arquivos automaticamente.

NÃO refatorar código automaticamente.

NÃO remover arquivos automaticamente.

NÃO criar novas arquiteturas automaticamente.

Primeiro faça uma análise detalhada e gere um relatório.

Toda recomendação deve conter:

- Problema encontrado
- Nível de severidade
- Impacto
- Arquivos afetados
- Sugestão de correção
- Risco da alteração

ITENS A ANALISAR

1. Estrutura do Projeto

Verificar:

- Arquivos vazios
- Rotas órfãs

- Componentes não utilizados
- Imports não utilizados
- Hooks não utilizados
- APIs não utilizadas
- Duplicação de funcionalidades
- Pastas obsoletas
- Código morto

Classificar cada item como:

- Seguro remover
 - Requer validação manual
 - Não remover
-

2. Arquitetura Next.js

Validar:

- App Router
- Layouts
- Loading states
- Error boundaries
- Route groups
- Middleware/Proxy
- Dynamic routes
- Server Components
- Client Components

Identificar:

- Más práticas
 - Arquivos incorretos
 - Estruturas duplicadas
 - Rotas conflitantes
-

3. Segurança

Analisar:

- Exposição de chaves
- Variáveis de ambiente
- Supabase Keys
- APIs públicas
- Uploads de arquivos
- XSS
- CSRF
- SQL Injection
- Validação de entrada

- Sanitização de dados

Classificar riscos:

- Crítico
 - Alto
 - Médio
 - Baixo
-

4. Banco de Dados (Supabase)

Validar:

- Queries ineficientes
- N+1 queries
- Índices ausentes
- Tabelas sem uso
- Campos não utilizados
- Estrutura das migrations
- RLS (Row Level Security)
- Policies

Informar:

- O que pode causar lentidão
 - O que pode causar vazamento de dados
-

5. Performance

Analisar:

- Re-renderizações desnecessárias
- Componentes pesados
- Bundle size
- Imports desnecessários
- Imagens
- Cache
- Memoização
- Lazy loading

Gerar lista priorizada:

- Ganho alto
 - Ganho médio
 - Ganho baixo
-

6. Escalabilidade SaaS

Considerando que o sistema será utilizado por múltiplos restaurantes.

Verificar:

- Multi-tenant
- Isolamento de dados
- Estrutura por empresa
- Escalabilidade do Supabase
- Escalabilidade do frontend
- Custos futuros

Identificar gargalos antes de 10, 50, 100 e 500 clientes.

7. Qualidade de Código

Verificar:

- Componentes grandes demais
 - Duplicação de lógica
 - Falta de abstração
 - Nomeação inconsistente
 - Organização de pastas
 - Tipagem
 - Tratamento de erros
-

8. Deploy e Produção

Validar:

- Compatibilidade com Vercel
 - Build warnings
 - Middleware deprecated
 - Variáveis obrigatórias
 - Configurações de produção
-

FORMATO DA RESPOSTA

Responder apenas em formato de relatório.

NÃO alterar código.

NÃO gerar commits.

NÃO criar arquivos.

NÃO refatorar automaticamente.

Primeiro identificar todos os problemas.

Depois gerar uma lista priorizada:

P0 = Crítico P1 = Alto P2 = Médio P3 = Baixo

Somente após o relatório completo sugerir um plano de correção em etapas.